

STAT 240

Week 2: More R Background

Owen G. Ward

This week

- Some more details on the hot hand (a simple data science example)
- More R review
 - Reading data/files into R
 - Some plotting
 - Relaxed pace, please ask questions if you have them!
- This course does not assume you know R, so there is time built in for any/all questions!

Labs

- Lab this week just set up, making sure you can submit the required pdf
- Next week first problem set posted, attendance taken
- Questions about logistics?

Office Hours

- My office hours will be in K10561
- Wednesday 2.30-3.15pm (Straight after class)
- Thursday 3.00-3.45pm
- Common for 240/260, let me know if you can't make either time

Returning to the hot hand

- Last time, we examined the “hot hand” hypothesis, and did one revolution around the wheel of science:

```
data <- read.csv("shot_logs.csv")

data <- data[order(
  data$player_name, data$GAME_ID,
  data$PERIOD, data$SHOT_NUMBER
), ]

W <- 0
X <- 0
Y <- 0
Z <- 0
...
```

Scoring percent after making a shot: 44.8%

Scoring percent after missing a shot: 45.5%

Hot hand, part 2

- We found the scoring percent after making a shot was in fact smaller than the scoring percent after missing a shot. There are several problems with our analysis, and more questions that emerge.
- This is common when you do data science!
 - Discuss your initial findings with experts, revise and iterate

-
- The most immediate is that the organization of our `for` loop is incorrect



—“Vince Carter (blue) tries to make a layup shot, while Pau Gasol (yellow) defends the basket”

Hot hand, part 2

- Suppose the sorted shot log looks like this:

PLAYER	GAME.ID	SHOT.NUMBER	SHOT.RESULT
james harden	1	1	MADE
stephen curry	2	1	MADE
stephen curry	2	2	MISSED

-
- For this shot log and our *week01* code `hothand.r`, the first shot made by Stephen Curry in the second game is incorrectly counted as a successful shot by the player for which their immediately previously shot was successful, yielding this incorrect table: (**Discussion:** What is the correct table?)

	MADE	MISSED
HOT	1	1
NOT	0	0

Hot hand, part 2

- We need to do the tabulation one player at a time, one game at a time. This is done in `hothand2.r` by adding additional logic, cleaning up the mistake:

```

...
W = 0; X = 0; Y = 0; Z = 0
for (i in 2:dim(data)[1]) {
  if (data$GAME_ID[i - 1] == data$GAME_ID[i] &&
      data$player_name[i - 1] == data$player_name[i]) {
    if (data$SHOT_RESULT[i - 1] == "made") { # HOT
      ...
    }
  }
}

```

Scoring percent after making a shot: 44.8%

Scoring percent after missing a shot: 45.9%

- The result is essentially unchanged, which was expected as only a small fraction of shots are affected by the mistake

To think about

- Suppose you were asked how many shots were affected. How could you compute this?
- Type of thing a manager could ask, handy to have the number ready (or know how to get it!)

Hot hand, part ?

- **Discussion:** Now that our code is correct, is this evidence of a “cold” hand ($45.9\% > 44.8\%$, is the difference of 1.1% *significant*)? If we were to do another revolution around the wheel of science, how could we improve this analysis?
- Only the 2014-2015 season is examined, and all players are examined. Maybe the hot hand is season specific? Maybe it is player specific?

-
- We’ve defined a hot hand as an improved chance of “making” the next shot, after each “made” shot.
 - The hot hand as a concept does not have a precise definition (leading us to start by making a candidate definition).
 - Maybe the hot hand would emerge after a less conservative definition (for example, we might examine if “making” 3 or more out of the 5 previous shots improves the percent chance of “making” the next shot)
 - Any mistakes in the assumptions, or the code?

Hot hand, significance

- Just because two percentages differ doesn't mean there's evidence that the underlying processes differ
- Consider this example: I flip coin *A* 7 times and it comes up heads 5 times (~71% of the time). I flip coin *B* 10 times and it comes up heads 3 times (30% of the time).
Discussion: Is this evidence that coin *A* will come up heads more often (in future flips)?

	HEADS	TAILS
Coin <i>A</i>	5	2
Coin <i>B</i>	3	7

Hot hand, significance

- We can figure out if data are evidence for something by using *p*-values and null hypotheses.
- For the coins, the null hypothesis we will use is that coin *A* and coin *B* come up heads the same percentage of the time.
- With this null hypothesis, we can use the *Fisher exact test* to find a *p*-value:

```
example <- data.frame(HEADS = c(5, 3), TAILS = c(2, 7))
rownames(example) <- c("A", "B")
fisher.test(example)$p.value
```

```
[1] 0.1534348
```

```
example <- data.frame(HEADS = c(5, 3), TAILS = c(2, 7))
rownames(example) <- c("A", "B")
fisher.test(example)$p.value
```

```
[1] 0.1534348
```

- The *p*-value is 0.15. This means that if the null hypothesis were true (if coin *A* and coin *B* truly come up heads the same percentage of the time), then 15% of the time, the data would look “as or more extreme” than what was observed

Fisher's Exact Test

- One of the oldest and simplest hypothesis tests
- First developed by Ronald Fisher, an extremely influential but controversial figure
- Credited with founding much of modern statistics

The exact test

- Can actually get the distribution of the counts in the table here, conditional on the marginals (the row sums)
 - How many times you flip each coin
- Means we can compute the probability of the table observed, under the null hypothesis
 - Similar to a calculation you may have seen in Stat 270
- Along with any table that is as or more extreme!

-
- Any test you have seen before, such as a t-test, is an approximation using the central limit theorem
 - This test is exact, doesn't need a large sample to give a valid result

A/B Testing

- While this test might seem simple, it's extremely useful in Data Science!
- Commonly seen in industry (tech, advertising, etc) as an A/B test

A/B Testing

- Suppose you want to see if adding a photo to your promo email leads to more people clicking on the link
- Send different (random) people the same email, some have the photo, some don't (no other difference)
- Compute the proportion who click the link in each setting (**click through rate**)
- Then can use this test to see if the photo leads to more clicks, with statistical guarantees

A/B Testing and Causal Inference

- This might seem like a very simple setting, but it is in fact the simplest form of **Causal Inference**
- This is the holy grail of scientific understanding. Does A cause B? In this example, does an image in an email cause more people to go to a website?

Causal Inference

- We are interested in knowing a **potential outcome**.
- Would someone be more likely to click on the link, if they had seen the image?
- We never observe this
- Causal inference allows us to reason about questions like this carefully
- A key requirement is that the email (treatment) received is random for each person, which is only possible in idealized problems

Hot hand, significance

- If the p -value were really small (generally, significance starts at $p < 0.05$), it would mean that if the null hypothesis were true, the chance of seeing the sort of data that was seen is really small:
- That's evidence *against* the null hypothesis. For coins A and B in this example, there's no statistical evidence that the percentage of heads differ

-
- Let's look at a p -value for our hot hand experiment:

```
result <- data.frame(MADE = c(W, Y), MISSED = c(X, Z))
fisher.test(result)$p.value
```

```
[1] 0.0004026706
```

- Interesting: This is in fact statistical evidence for a “cold” hand
- What is our null hypothesis here? Other potential issues with our analysis can impact this also

Hot hand, player specific

- Taking a step back, let's consider the other discussion point and examine hot hand in a single player, Steph Curry
- Do we expect to see the same result when we look at a subset of our data instead?

-
- We subset to shots for this player:

```
...
# Examines hot hand for Stephen Curry.
data <- read.csv("shot_logs.csv")
data <- data[data$player_name == "stephen curry", ]
data <- data[order(
  data$GAME_ID, data$PERIOD,
  data$SHOT_NUMBER
), ]

W <- 0
X <- 0
Y <- 0
Z <- 0
for (i in 2:dim(data)[1]) {
  if (data$GAME_ID[i - 1] == data$GAME_ID[i]) {
    ...
  }
}
cat(sprintf("p-value: %.2e\n", fisher.test(result)$p.value))
```

```
...
# Examines hot hand for Stephen Curry.
data <- read.csv("shot_logs.csv")
data <- data[data$player_name == "stephen curry", ]
data <- data[order(
  data$GAME_ID, data$PERIOD,
  data$SHOT_NUMBER
), ]

W <- 0
X <- 0
Y <- 0
```

```
Z <- 0
for (i in 2:dim(data)[1]) {
  if (data$GAME_ID[i - 1] == data$GAME_ID[i]) {
    ...
  }
  cat(sprintf("p-value: %.2e\n", fisher.test(result)$p.value))
}
```

Scoring percent after making a shot: 44.5%
 Scoring percent after missing a shot: 52.2%
 p-value: 2.03e-02

Hot hand, player specific

```
...
data <- data[data$player_name == "stephen curry", ]
...
```

- This line of code subsets the rows of `data` using *logical indexing*
- Here `data$player_name == "stephen curry"` is a vector of TRUE or FALSE values with length given by the number of rows in `data`
- Rows are in the subset if their corresponding value in the vector is TRUE

Logical indexing

```
a <- c(6, 24, 96, 384, 1536)
b <- c(TRUE, TRUE, FALSE, TRUE, FALSE)
print(a[b])
```

```
[1] 6 24 384
```

- Logical indexing works for vectors as well as data frames

```
a <- c(6, 24, 96, 384, 1536)
print(a > 25)
```

```
[1] FALSE FALSE TRUE TRUE TRUE
```

```
print(a[a > 25])
```

```
[1] 96 384 1536
```

Logical indexing

Again, here is what it looks like for the example shot log from the first slides:

```
example <- data.frame(PPLAYER = c("james harden", "stephen curry", "stephen curry"),
                      GAME.ID = c("1", "2", "2"),
                      SHOT.NUMBER = c("1", "1", "2"),
                      SHOT.RESULT = c("MADE", "MADE", "MISSED"))
print(example$PPLAYER == "stephen curry")
```

```
[1] FALSE TRUE TRUE
```

```
example <- example[example$PPLAYER == "stephen curry", ]
```

	PLAYER	GAME.ID	SHOT.NUMBER	SHOT.RESULT
2	stephen curry	2	1	MADE
3	stephen curry	2	2	MISSED

Displaying this table

- R has some functions for displaying data in a nicer format
- Here we are using the `kable` function from the `knitr` package

```
kable(example)
```

	PLAYER	GAME.ID	SHOT.NUMBER	SHOT.RESULT
2	stephen curry	2	1	MADE
3	stephen curry	2	2	MISSED

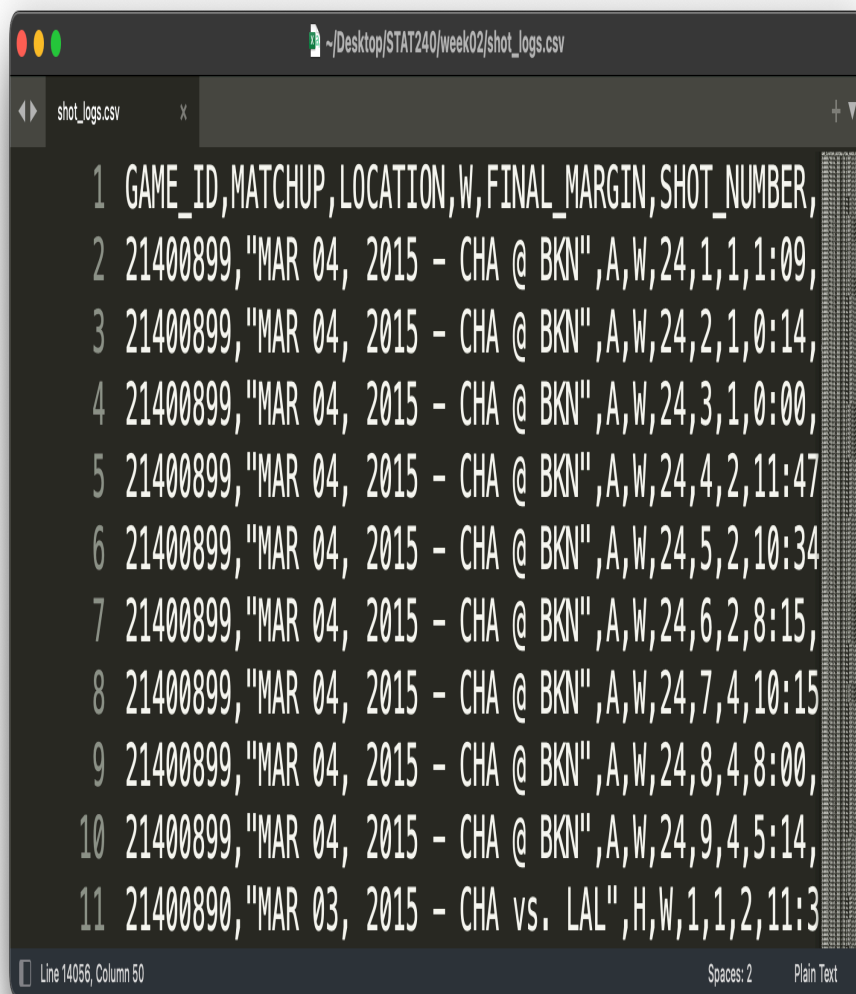
R packages

- We can install these packages locally using `install.package("package_name")`
- Then these packages need to be loaded when used using `library(package_name)`
- Note the use/absence of “ ” here
 - Need “ ” when installing packages
 - Don't need it when loading packages

Reading Files

Reading flat files

- Moving on: The first step of `hothand.r` and our extensions is reading in a file. What does this mean, and how do we do it?



```
1 GAME_ID,MATCHUP,LOCATION,W,FINAL_MARGIN,SHOT_NUMBER,
2 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,1,1,1:09,
3 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,2,1,0:14,
4 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,3,1,0:00,
5 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,4,2,11:47
6 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,5,2,10:34
7 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,6,2,8:15,
8 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,7,4,10:15
9 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,8,4,8:00,
10 21400899,"MAR 04, 2015 - CHA @ BKN",A,W,24,9,4,5:14,
11 21400890,"MAR 03, 2015 - CHA vs. LAL",H,W,1,1,2,11:3
```

-
- Here we have the shot log *comma separated variable* file. We read it with:

```
data <- read.csv("shot_logs.csv")
```

Reading flat files

- A file is a sequence of bytes
- Bytes are 8-bit values (they take the range 0 - 255)
- According to the “ASCII” table (American Standard Code for Information Interchange), the range of values 32 - 126 contains the Latin alphabet without accents and punctuation (the range 128 - 165 contains accented Latin letters)
- Some Greek letters appear in the range 224 - 235

-
- [Unicode](#) can be used to specify characters in a long list of scripts (Cyrillic, Arabic, Ethiopic, ... “Unicode covers all the characters for all the writing systems of the world, modern and ancient.”)
 - A unicode character is a short sequence of bytes known as a code point

Reading flat files

- Some files (`.png` for images, `.xlsx` for spreadsheets, `.pdf` for documents) use their own format for the meaning and layout of the byte values in the file
- A file is called a flat file if it uses the ASCII or Unicode encoding (we can read the file if we open it in a “text editor”—A program such as *notepad* that displays the ASCII or Unicode characters on the screen)

Filesystems

- Every file resides in a directory. When we do `read.csv("shot_logs.csv")`, R Studio will look in its current working directory for a file named `shot_logs.csv` and give an error if that file doesn't exist
- You can change R Studio's working directory using the function `setwd` or using the file browser in the *Files* tab in the lower right pane of R Studio

Files and folders

- Understanding filesystems and folders are an important (but boring) skill for any data scientist
- For large datasets, might not be able to open the file directly (what if there are millions/billions of observation)
- Might not even have the data on your own machine
 - Stored on company server which you can read, but only from the command line
 - This will be a Unix system

The R working directory

- The **working directory** is where the analysis in a script/markdown document takes place
- By default, this is where R will look for data files and save output
- This will depend on where the script is itself saved and some other settings
- Can see this on the console or using `getwd()`

Filepaths

- If you want to access something outside of your working directory you need to tell R where it is
- For example, if I wanted to access `shot_logs.csv` from anywhere on my computer I would need the **absolute path** to it

```
data <- read.csv("/Users/owen/Documents/Github/stat_240/shot_logs.csv")
```

```
data <- read.csv("/Users/owen/Documents/Github/stat_240/shot_logs.csv")
```

- Can use `/` for both Windows and Mac in R
- Note if anyone else ran this code it wouldn't work, unique to my machine
- This is a problem when you want to share code
- Thankfully there are better ways of doing this!

R Studio Projects

- RStudio provides what are known as Projects
- You should set one up for this course, and place all of your code, files inside of it!
- When you open an RStudio Project the working directory for all scripts in that project is where the project is saved
- Only need to worry about relative paths inside the project

Relative Paths

- I have `shot_logs.csv` inside a folder in my project
- I only need to tell R the path inside the project to open it

```
data <- read.csv("slides/week_02/shot_logs.csv")
```

- If you had a copy of this project (set up correctly) this code would run on your computer too!

Reading files in a markdown document

- When you run code inside a qmd file (even inside a project) the working directory is where that file is saved!
- So to correctly load in the data in this file

```
## as this file in slides/week_02/ already  
data <- read.csv("shot_logs.csv")
```

Reading flat files

- Flat files come in many formats (the only defining characteristic is they can be read by eye in a text editor). Here's a *tab separated variable file* containing historical stock prices for 27 NASDAQ stocks on November 17th, 2015 (this file `AS-N100.tsv` is provided in the folder for this lecture):

The screenshot shows a text editor window titled "AS-N100.tsv" with the following content:

	date	timestamp	ticker	open	high	low	close	volume
1	2015-11-17	09:31:00	LLTC	44.17	44.17	44.17	44.17	4
2	2015-11-17	09:32:00	LLTC	44.16	44.18	44.12	44.14	1
3	2015-11-17	09:33:00	LLTC	44.16	44.18	44.09	44.09	1
4	2015-11-17	09:34:00	LLTC	44.05	44.09	43.95	44.01	6
5	2015-11-17	09:35:00	LLTC	44.03	44.07	43.95	43.95	1
6	2015-11-17	09:36:00	LLTC	43.99	44.05	43.96	44.05	4
7	2015-11-17	09:37:00	LLTC	44.06	44.13	44.02	44.08	1
8	2015-11-17	09:38:00	LLTC	44.1	44.15	44.1	44.14	5
9	2015-11-17	09:39:00	LLTC	44.13	44.19	44.13	44.17	1
10	2015-11-17	09:40:00	LLTC	44.16	44.21	44.14	44.21	4
11	2015-11-17	09:40:00	LLTC	44.16	44.21	44.14	44.21	4

Reading flat files

- When we do `read.csv('AS-N100.tsv')` we get a data frame with a single column of strings, containing the file line by line. That's because `read.csv` expects the values for each record to be separated by commas
- We can tell R how to read a flat file by using additional parameters to the `read.csv` call:

```
data <- read.csv("AS-N100.tsv", sep = "\t")
print(dim(data))
```

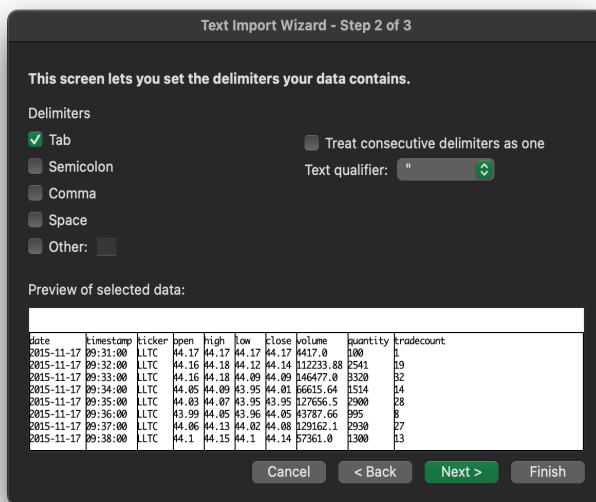
```
[1] 10000    10
```

Reading flat files

- Note that `read.csv` is a version of the function `read.table` that differs only in the default values for the parameters (`sep = ','` for example)
- You can read the help file for the function `read.table` to understand how to read in a specific file (dealing with quotations, row numbers and so on).
- In the console type `??read.table` and press Enter. This will open a help file

Reading flat files in excel

- If you open a flat file in excel, you'll (probably) be prompted for information to correctly read the file
- Skimming flat files in excel or text editors is a great way to get a preliminary understanding of the data



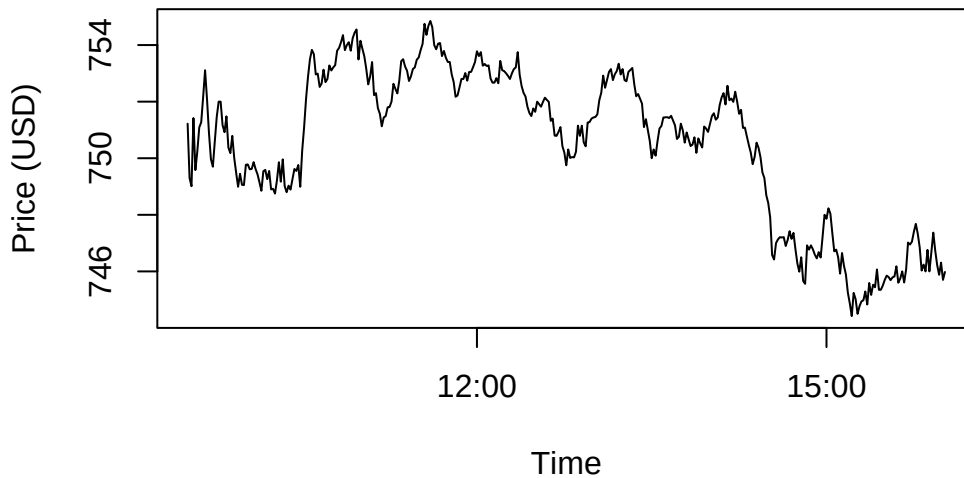
Preview of plotting

- Basic plots can be made with the `plot` function in R. Plots will appear in the *Plots* tab of the lower right pane of R Studio
- On the next slide we plot the opening price for GOOGL for each minute on November 17th, 2015

```
subset <- data[data$ticker == "GOOGL", ]
xs <- as.POSIXct(strptime(paste(
  "2015-11-15",
  subset$timestamp
), "%Y-%m-%d %H:%M:%S"))

ys <- subset$open
plot(xs, ys,
     xlab = "Time", ylab = "Price (USD)",
     main = "GOOGL open price / minute", type = "l"
)
```

GOOGL open price / minute



Example: NASDAQ

- Five basic summary statistics: mean, median, standard deviation, min, max. In R these are the functions `mean`, `median`, `sd`, `min`, `max`

```
mu <- mean(subset$open)
print(mu)
```

```
[1] 750.4808
```

Example: NASDAQ

- Five basic summary statistics: mean, median, standard deviation, min, max. In R these are the functions `mean`, `median`, `sd`, `min`, `max`

```
sigma <- sd(subset$open)
print(sigma)
```

```
[1] 2.70453
```

Summary + Reading

- Optional: Cotton Chapter 3
- Next week
 - Statistics review, slightly more advanced R programming
- Submit lab0 to ensure you won't have issues with future problem sets
- Office hours starting this week!